

100 Drone Swarm Simulation — Detailed Team Plan

PDC Project · Spring 2026 · 4-Member Team

Big Picture:

We simulate **100 autonomous drones** in a decentralized swarm. No central brain — each drone senses local neighbors and acts on simple rules (Boids) that produce complex, emergent behavior.

Phases:

→ **2D (Python)** to prove logic → **3D Ursina** for rich visuals → **Gazebo (ROS2)** for real-robot-grade realism.

Team Roles

Member	Role	Primary Domain
Member A	Physics & Environment Engine	World setup, simulation loop, collision physics, dynamic obstacles
Member B	Swarm Behavioral Logic	Boids rules, steering forces, task allocation, fault tolerance
Member C	Visualization & UI/UX	Rendering (2D→Ursina→Gazebo), camera, dashboard, metrics display
Member D	Optimization & Parallelism	Spatial indexing, multiprocessing, performance logging, benchmarking

Technology Progression

Phase 1 (2D)	Phase 2 (3D – Ursina)	Phase 3 (3D – Gazebo)
Python + Pygame NumPy kinematics Quadtree (2D) Multiprocessing CSV/JSON logs	→ Python + Ursina Engine 3D spheres / drone mesh Octree / Voxel Grid GPU-ready (VisPy optional) Real-time 3D dashboard	→ ROS2 + Gazebo Harmonic URDF drone models ROS2 topics / services PX4/MAVROS SITL ROS2 bag recording

MILESTONE 1 — Environment & Foundation

Goal: Spawn 100 drones in a 2D plane. They move, sense neighbors, and don't crash into walls.

Duration: Weeks 1–3 | **Trigger to proceed:** 100 drones run for 1,000+ ticks with zero wall collisions.

Member A — Physics & Environment Engine

Task ID	Task	Details	How-To
A1.1	Create 2D world canvas	1000×1000 unit continuous plane. Define boundary walls with configurable: wrap-around (toroidal) OR hard-wall repulsion.	<code>pygame.display.set_mode((1000,1000))</code> + wall constants
A1.2	Static obstacle placement	Place 5–10 rectangular obstacles at fixed positions. Store as list of <code>Rect(x,y,w,h)</code> .	<code>pygame.draw.rect()</code> per obstacle
A1.3	Main simulation loop	<code>while running: handle_events() → update_physics() → render()</code> . Fixed time step $\Delta t = 0.02s$ (50 FPS target).	Python <code>time.perf_counter()</code> for tick timing
A1.4	Wall & obstacle repulsion	When drone within <code>d_wall=20</code> of boundary, apply outward repulsion force $F = k / dist^2$. Bounce as fallback.	Override velocity component on boundary hit
A1.5	Simulation config file	Create <code>config.yaml</code> — world size, num_drones, dt, seed, obstacle list. Load at startup.	PyYAML or json

Member B — Swarm Behavioral Logic

Task ID	Task	Details	How-To
B1.1	Drone class (2D)	Properties: <code>id</code> , <code>pos [x,y]</code> , <code>vel [vx,vy]</code> , <code>heading θ</code> , <code>state</code> . Methods: <code>update_position(dt)</code> .	NumPy arrays for pos/vel
B1.2	Kinematic update	<code>pos += vel * dt</code> . Clamp speed: <code>if vel > v_max: vel = vel/ vel * v_max</code> .	NumPy vector ops
B1.3	Random initialization	Scatter 100 drones randomly in world. Use <code>random.seed(config.seed)</code> for reproducibility.	<code>np.random.uniform(0, world_size, (100,2))</code>
B1.4	Basic heading display	Draw a short line in the direction of <code>vel</code> from each drone's center.	<code>pygame.draw.line()</code> with angle from <code>atan2(vy,vx)</code>
B1.5	Neighbor list structure	Each drone holds a <code>neighbors[]</code> list updated every tick before behavior computation.	Share global positions array; B1 reads slice

Member C — Visualization & UI/UX

Task ID	Task	Details	How-To
---------	------	---------	--------

Task ID	Task	Details	How-To
C1.1	Pygame window + render loop	Create 800×600 window. Clear → draw obstacles → draw drones → flip. Target 60 FPS.	<code>pygame.init(), clock.tick(60)</code>
C1.2	Draw 100 drones	Each drone = small circle (radius 4px). Use <code>drone.id</code> for color grouping (optional).	<code>pygame.draw.circle(surf, color, pos, 4)</code>
C1.3	Draw obstacles	Filled grey rectangles for static obstacles.	<code>pygame.draw.rect()</code>
C1.4	FPS counter overlay	Top-left corner: real-time FPS display.	<code>pygame.font.render(f"FPS: {clock.get_fps():.0f}")</code>
C1.5	Pause/Play button	Press <code>SPACE</code> to pause/resume simulation. Draw button state on HUD.	Boolean <code>paused</code> flag in event loop

Member D — Optimization & Parallelism

Task ID	Task	Details	How-To
D1.1	Naïve $O(N^2)$ neighbor check	For each drone, check all others. Store pairs where <code>dist < R</code> . This is the BASELINE for benchmarking.	Double nested loop over drone list
D1.2	Measure baseline FPS	Log FPS every 100 ticks to CSV: <code>tick, fps, num_drones</code> . Run with $N=50, 100, 200$.	<code>benchmark_log.csv</code>
D1.3	Quadtree implementation	Implement (or use library) a Quadtree. Insert all drone positions each tick. Query radius R per drone.	<code>QuadTree</code> class or <code>scipy.spatial.KDTree</code>
D1.4	Compare $O(N^2)$ vs Quadtree	Benchmark both at 100 drones. Document FPS difference.	Same CSV logging, compare in <code>pandas</code>
D1.5	Shared positions array	Create <code>np.ndarray</code> of shape <code>(100, 2)</code> in shared memory. All drones read from it each tick.	<code>multiprocessing.shared_memory</code> module

 MILESTONE 2 — Decentralized Coordination Algorithms

Goal: Drones flock, self-organize, and dynamically allocate tasks — all without a central controller.
Duration: Weeks 4–6 | **Trigger to proceed:** Visible flocking + task assignment with no global coordinator.

Member A — Physics & Environment Engine

Task ID	Task	Details	How-To
---------	------	---------	--------

Task ID	Task	Details	How-To
A2.1	Predictive collision avoidance	Project drone positions $k=3$ ticks ahead. If two projected paths intersect within d_safe , apply early velocity adjustment.	Linear extrapolation: $pos_pred = pos + vel * k * dt$
A2.2	Drone–obstacle repulsion	If drone within $d_obs=30$ of any obstacle edge, apply inverse-square repulsive force.	$F_obs = - (obs_nearest_pt - pos) / dist^2$
A2.3	Drone–drone safe distance	Layer 1 (reactive): if $dist < d_safety=15$, override all steering with a pure repulsion vector.	Priority override on a_total computation
A2.4	Dynamic obstacle (optional)	Add 1–2 moving rectangles that follow a looping path. Update obstacle position in main loop.	$obs_pos += obs_vel * dt$; bounce on world edges
A2.5	World config tuning	Expose d_safe , d_wall , v_max , a_max in config file. Allow hot-reload via file watch.	<code>watchdog</code> library or manual reload key

Member B — Swarm Behavioral Logic

Task ID	Task	Details	How-To
B2.1	Separation force	Steer away from neighbors closer than r_sep . $F_sep = -\sum (pos_j - pos_i) / \Delta pos ^2$ for $ \Delta < r_sep$.	NumPy vectorized sum over neighbor list
B2.2	Alignment force	Match average velocity of neighbors. $F_align = mean(vel_neighbors) - vel_i$.	<code>np.mean(neighbor_vels, axis=0)</code>
B2.3	Cohesion force	Steer toward center of neighbor positions. $F_coh = mean(pos_neighbors) - pos_i$.	<code>np.mean(neighbor_pos, axis=0)</code>
B2.4	Combined steering	$a_i = w_s * F_sep + w_a * F_align + w_c * F_coh + w_o * F_obs$. Clamp $ a < a_max$. Apply: $vel += a * dt$.	Tunable weights from config
B2.5	Decentralized task allocation	Tasks = waypoints/zones in the world. Drone in SEARCHING state claims nearest unclaimed task. Uses auction : within msg radius, compare distance costs, lowest wins.	State machine: SEARCHING → ENGAGING → RETURNING
B2.6	Message-passing protocol	Each tick, drone broadcasts $\{id, pos, vel, task_id, status\}$ to neighbors within R. Process incoming to detect conflicts.	Python dict passed with neighbor data

Task ID	Task	Details	How-To
B2.7	Consensus: task conflict resolution	If two drones claim same task: drone with lower <code>min_cost</code> (or lower <code>id</code> as tiebreaker) retains it; other releases and searches again.	Compare <code>cost</code> field in received messages

Member C — Visualization & UI/UX

Task ID	Task	Details	How-To
C2.1	Sliders for flocking weights	4 on-screen sliders: <code>w_sep</code> , <code>w_ali</code> , <code>w_coh</code> , <code>w_obs</code> . Drag to adjust in real-time.	<code>pygame_gui</code> slider widget or custom slider
C2.2	Neighbor link visualization	When <code>DEBUG_LINKS=True</code> , draw a thin grey line from each drone to each of its neighbors.	<code>pygame.draw.line(surf, GREY, pos_i, pos_j, 1)</code>
C2.3	Velocity vector arrows	Small arrow from each drone in direction of current velocity (scaled by speed).	<code>pygame.draw.line()</code> + arrowhead polygon
C2.4	Task zone display	Draw task zones as colored circles/rects. Color: grey=unclaimed, yellow=assigned, green=completed.	Per-task state drives color
C2.5	Drone state color coding	<code>SEARCHING</code> =blue, <code>ENGAGING</code> =orange, <code>RETURNING</code> =green, <code>IDLE</code> =grey.	Lookup dict from state string to RGB
C2.6	Real-time metrics HUD	Top HUD bar: <code>Tasks Done: X/Y Collisions: N FPS: Z Tick: T</code> .	<code>pygame.font.render()</code> with live values

Member D — Optimization & Parallelism

Task ID	Task	Details	How-To
D2.1	Spatial Hash Grid	Divide world into cells of size <code>R</code> . Each tick: clear grid, insert all drones, query only 9 adjacent cells per drone. $O(1)$ average-case.	Dict keyed by <code>(cell_x, cell_y)</code>
D2.2	KDTree integration	<code>scipy.spatial.KDTree(positions).query_ball_point(pos_i, r=R)</code> per drone. Compare vs grid.	from <code>scipy.spatial</code> <code>import KDTree</code>
D2.3	Parallel drone updates	Split 100 drones into 4 chunks. Use <code>multiprocessing.Pool(4).map(update_chunk, chunks)</code> . Merge results.	<code>pool.starmap()</code> with chunk slices

Task ID	Task	Details	How-To
D2.4	Shared memory for positions	Use <code>multiprocessing.shared_memory.SharedMemory</code> to share drone position array across processes. Avoid pickling overhead.	<code>np.ndarray</code> backed by shared memory block
D2.5	Performance report (M2)	Compare: Sequential+Naive / Sequential+Grid / Parallel+Grid at N=100. Log FPS, CPU%.	Pandas DataFrame → bar chart via Matplotlib

 MILESTONE 3 — Mission Execution & Full 3D Upgrade

Goal: Full missions run, drones survive failures, and we graduate to 3D (Ursina → Gazebo).

Duration: Weeks 7–9 | **Final Deliverable:** 3D Gazebo simulation + performance report.

Member A — Physics & Environment Engine

Task ID	Task	Details	How-To
A3.1	Dynamic 3D obstacles	Moving spheres/cubes in 3D space following sinusoidal/random paths. Drones must dodge in real-time.	Update <code>obs3d.pos += obs3d.vel * dt</code> ; Ursina <code>Entity</code> position
A3.2	Mission scenario: Area Coverage	Divide 3D volume into voxel cells. Track which cells any drone has visited. Goal: 95% coverage. End condition: time limit or 95% reached.	3D grid of boolean flags
A3.3	Fault injection system	Randomly remove N drones (10% or 25%) at a specified simulation tick. Log event. Task recovery should kick in within 5 ticks.	<code>del drones[random_id]</code> + heartbeat timeout trigger
A3.4	URDF drone model (Gazebo)	Export/obtain a simple quadrotor URDF model. Load in Gazebo world. Define rotor links, collision boxes, sensor plugins.	<code>quadrotor.urdf.xacro</code> from PX4 or MAVSDK templates
A3.5	Gazebo world file	Create <code>swarm_world.world</code> (SDF format) with ground plane, 3D obstacles as boxes/cylinders. 100m×100m×50m airspace.	Gazebo <code><world></code> XML/SDF syntax

Member B — Swarm Behavioral Logic

Task ID	Task	Details	How-To
---------	------	---------	--------

Task ID	Task	Details	How-To
B3.1	Full 3D kinematics	Extend state to $[x, y, z, vx, vy, vz]$. Update: $\text{pos3d} += \text{vel3d} * dt$. Distance: $d = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$. Boundary = axis-aligned 3D box.	NumPy 3D vectors
B3.2	3D flocking forces	Separation/Alignment/Cohesion forces now in 3D. Separation must also consider vertical (z-axis) crowding.	Extend 2D force formulas to 3D vectors
B3.3	Target seeking behavior	New force: $F_{\text{target}} = (\text{target_pos} - \text{pos_i}) / \Delta $. Used in ENGAGING state for task completion.	Add to combined steering formula
B3.4	ROS2 behavior node	Wrap swarm logic in a ROS2 Python node (<code>rclpy</code>). Subscribe to <code>/drone_N/odom</code> ; publish to <code>/drone_N/cmd_vel</code> .	<code>rclpy.node.Node</code> + publisher/subscriber
B3.5	Fault-tolerant re-clustering	On drone removal, remaining drones detect absence via heartbeat timeout. Re-run task allocation auction for orphaned tasks.	Heartbeat counter decrement each tick; release on zero

Member C — Visualization & UI/UX

Task ID	Task	Details	How-To
C3.1	3D Ursina setup	Install Ursina: <code>pip install ursina</code> . Create <code>App()</code> , add a 3D box boundary wireframe, sky. Add <code>EditorCamera()</code> for orbit/pan/zoom.	<code>from ursina import *; app = Ursina()</code>
C3.2	Render 100 drone entities	Each drone = <code>Entity(model='sphere', color=color.cyan, scale=0.5)</code> . Update <code>entity.position = Vec3(*drone.pos)</code> each tick.	Ursina <code>update()</code> loop
C3.3	Trail effects	Attach particle trail or manually draw last N positions per drone as fading spheres.	Store <code>deque(maxlen=20)</code> of past positions
C3.4	3D Ursina UI overlay	Text panels for FPS, tick, mission progress, drone count. Ursina <code>Text()</code> entities anchored to screen corners.	<code>Text('FPS: ...', position=(-0.85, 0.45))</code>
C3.5	Gazebo visualization	Launch Gazebo with <code>rviz2</code> side panel. Configure RViz2 to display <code>/drone_N/pose</code> topics as arrow markers. Color by state.	RViz2 <code>MarkerArray</code> topic via ROS2
C3.6	Video recording	Record Ursina sessions using <code>ursina.application.screenshot()</code> per frame → compile to MP4 with <code>ffmpeg</code> .	<code>ffmpeg -framerate 30 -i frame_%04d.png output.mp4</code>

Member D — Optimization & Parallelism

Task ID	Task	Details	How-To
D3.1	Octree for 3D neighbor search	Extend Quadtree → Octree. 8 child nodes per node. Insert all 3D positions, query sphere of radius R.	Implement recursive Octree class or use <code>pykdtree</code>
D3.2	NumPy vectorized batch update	Replace all Python loops with matrix math: <code>positions += velocities * dt</code> — one line updates all 100 drones simultaneously.	<code>np.ndarray</code> broadcasting
D3.3	Ray framework (optional upgrade)	Replace <code>multiprocessing</code> with <code>Ray</code> for zero-copy shared memory and actor-based drone processes.	<code>pip install ray;</code> <code>ray.init(); @ray.remote</code>
D3.4	ROS2 parallel launch	In Gazebo phase: launch 100 drone nodes simultaneously via <code>ros2 launch swarm_bringup spawn_swarm.launch.py n:=100</code> .	<code>launch_ros.actions.Node</code> per drone in loop
D3.5	Final benchmarking suite	Measure: Sequential FPS vs Parallel FPS vs 3D Ursina FPS vs Gazebo update rate. Plot all in one comparison chart.	Pandas pivot table → seaborn bar chart
D3.6	Final performance report	Write <code>performance_report.md</code> : tables of metrics at N=25, 50, 75, 100 drones. Include robustness (% tasks completed at 10%/25% failure).	Markdown table + embedded plots

🔗 3D Simulation Pathway: Step-by-Step

Phase A — 3D with Ursina Engine (Start Here)

Why Ursina first?

Ursina is pure Python, zero ROS dependency, fast to prototype. We validate all 3D math and logic here before committing to Gazebo infrastructure.

Step 1: Install Ursina

pip install ursina

Step 2: Port drone class to 3D

- pos: (x, y, z) vel: (vx, vy, vz)

- distance: sqrt(dx²+dy²+dz²)

- boundary: x ∈ [0,W], y ∈ [0,H], z ∈ [0,D]

Step 3: Create Ursina scene

- App() with EditorCamera

- Box wireframe boundary (Entity with wireframe=True)

- 100 sphere Entities for drones

Step 4: Main update() function

- Called every frame by Ursina engine
- Run swarm logic → update entity.position per drone

Step 5: Add UI overlays, trails, debug lines

Step 6: Validate behavior (flocking in 3D, task allocation in 3D volume)

Reference:

- [Ursina Engine Docs](#)
- [Ursina Entity API](#)

Phase B — Full 3D with Gazebo + ROS2 (Final Phase)

Why Gazebo?

Gazebo offers physics-accurate simulation (gravity, aerodynamics if needed), URDF drone models, sensor simulation (cameras, lidar), and real ROS2 integration — making results transferable to real hardware.

Step 1: Install ROS2 Humble + Gazebo Harmonic
`sudo apt install ros-humble-desktop gazebo`

Step 2: Create ROS2 workspace
`mkdir -p ~/swarm_ws/src`
`cd ~/swarm_ws && colcon build`

Step 3: Create drone URDF model

- quadrotor.urdf.xacro
- Add: base_link, 4 rotor joints, IMU plugin, Odometry plugin
- Reference: PX4 SITL models or MAVSDK drone show models

Step 4: Create Gazebo world (SDF)

- swarm_arena.world: 100m×100m×50m airspace
- Static obstacles: box entities at defined positions
- Optional: moving obstacle with trajectory plugin

Step 5: Spawn 100 drones

- ROS2 launch file: loop spawn_entity.py for drone_0...drone_99
- Each gets unique namespace: /drone_0/, /drone_1/, ...

Step 6: Swarm behavior nodes (ROS2)

- Each drone runs: swarm_node.py
- Subscribes: /drone_N/odom (position + velocity)
- Publishes: /drone_N/cmd_vel (velocity commands)
- Logic: import and run Milestone 2 swarm algorithms

Step 7: Visualization

- Gazebo GUI: 3D view of all drones
- RViz2: arrow markers per drone colored by state
- ros2 bag record all topics for replay

Step 8: Run mission scenarios and record metrics

Key References:

- [MAVSDK Drone Show \(100-drone SITL\)](#)
- [PX4 SITL + Gazebo](#)
- [ROS2 Humble Docs](#)
- [Gazebo Harmonic SDF Reference](#)
- [ros2 spawn_entity for multiple robots](#)

 Evaluation Metrics (All Milestones)

Metric	Target	Measured In
Mission Completion Time	Minimize	Simulation ticks / wallclock seconds
Collision Count	0 (ideal)	Per 1000 ticks
Area Coverage %	> 95%	% of voxels visited
FPS (2D sequential)	≥ 60 FPS @ 100 drones	<code>pygame.clock.get_fps()</code>
FPS (2D parallel)	≥ 60 FPS @ 200 drones	Same
FPS (3D Ursina)	≥ 30 FPS @ 100 drones	Ursina FPS counter
Robustness Index	> 90% tasks done at 25% failure	After fault injection
Message Overhead	< 10 msgs/tick/drone	Logged in behavior node
Speedup (parallel vs seq)	≥ 2× at 4 cores	<code>time.perf_counter()</code>

 Repository Structure

```
drone-swarm/
├── src/
│   ├── core/
│   │   ├── drone.py           # Drone class (2D + 3D)
│   │   ├── environment.py     # World, obstacles, boundaries
│   │   ├── spatial_index.py   # Quadtree + Octree + Grid Hash
│   │   └── simulation.py      # Main simulation loop
│   ├── behaviors/
│   │   ├── flocking.py        # Separation, Alignment, Cohesion
│   │   ├── task_allocation.py # Greedy + Auction assignment
│   │   └── consensus.py       # Message-passing, conflict resolution
│   ├── visualization/
│   │   ├── renderer_2d.py     # Pygame 2D renderer
│   │   ├── renderer_3d.py     # Ursina 3D renderer
│   │   └── dashboard.py       # UI sliders, HUD, metrics
│   ├── parallel/
│   │   ├── multiproc_pool.py  # Multiprocessing pool updater
│   │   └── ray_actors.py      # Optional Ray actor implementation
│   └── ros2/
│       ├── swarm_node.py      # Per-drone ROS2 node
│       └── spawn_swarm.launch.py
```

```
|— gazebo/
|   |— models/
|   |   |— quadrotor/      # URDF + meshes
|   |   |— worlds/
|   |       |— swarm_arena.world # SDF world file
|— tests/
|   |— test_flocking.py
|   |— test_collision.py
|   |— test_task_allocation.py
|— docs/
|   |— project_blueprint.md
|   |— team_execution_plan.md
|   |— detailed_team_plan.md ← THIS FILE
|   |— research_summary.md
|— benchmarks/
|   |— benchmark_log.csv
|— config.yaml
|— requirements.txt
```



Dependencies

```
# Core
pip install numpy pygame scipy pyyaml

# 3D Ursina
pip install ursina

# Parallel
pip install ray

# Analysis
pip install pandas matplotlib seaborn

# ROS2 / Gazebo (Linux only – use WSL2 on Windows)
sudo apt install ros-humble-desktop
sudo apt install gazebo
pip install rclpy

# Optional: High-performance 3D
pip install vispy pyopengl
```



Week-by-Week Timeline

Week	Focus	Milestone	Lead
1	Repo setup, 2D world canvas, Drone class, naïve neighbor check, Pygame render	M1	All

Week	Focus	Milestone	Lead
2	Wall collision, static obstacles, spatial hash grid, FPS baseline logging	M1	A + D
3	Basic steering (random walk), Quadtree, reproducibility testing	M1 complete ✓	B + D
4	Separation + Alignment + Cohesion forces, weight sliders, neighbor viz	M2	B + C
5	Task allocation (proximity + auction), message-passing, consensus	M2	B + D
6	Predictive collision avoidance, parallel drone updates, M2 benchmark report	M2 complete ✓	A + D
7	3D port to Ursina: drone entity setup, EditorCamera, 3D boids	M3 (Ursina)	C + B
8	Fault injection, dynamic obstacles (3D), missions (area coverage), Ursina polish	M3 (Ursina) ✓	A + C
9	Gazebo world + URDF, spawn 100 drones, ROS2 swarm node, RViz2 markers	M3 (Gazebo)	All
10	Final benchmarks, video recording, performance report, LaTeX report	Final ✓	All

✓ Deliverables Checklist

- ☐ **Source Code** — GitHub repo with milestone branches ([m1](#), [m2](#), [m3-ursina](#), [m3-gazebo](#))
- ☐ **2D Demo Video** — Screen-recording of Milestone 2 flocking + task allocation
- ☐ **3D Ursina Demo** — Ursina simulation video with transparent boundary, trails, and HUD
- ☐ **3D Gazebo Demo** — Gazebo + RViz2 recording of 100 drones in mission
- ☐ **Benchmark Report** — [performance_report.md](#) with FPS tables and speedup charts
- ☐ **Final Report** — LaTeX/PDF covering architecture, algorithms, results, and analysis

🔗 Key References

Resource	Link	Used By
Reynolds Boids (1987)	Classic flocking paper	B
Ursina Engine	https://www.ursinaengine.org/	C
MAVSDK Drone Show (100-drone SITL)	https://github.com/alireza787b/mavsdk_drone_show	A, C
Drone Swarm (Python + Matplotlib)	https://github.com/jeanjerome/drone-swarms	B
PX4 SITL + Gazebo	https://docs.px4.io/main/en/sim_gazebo_gz/	A
ROS2 Humble Docs	https://docs.ros.org/en/humble/	All
vmodel (statistical data)	https://github.com/lis-epfl/vmodel	D
UAV-Swarm-Simulator	https://github.com/matteoprata/UAV-Swarm-Simulator	B

Document Version 1.0 — Created for PDC Project, Spring 2026

Team: 4 members | Simulation: 100 autonomous drones | Platforms: 2D Pygame → 3D Ursina → Gazebo (ROS2)